

Outfit Lab

The Tuesday story: build Git, one problem at a time.

Each lab dresses one character together. Every button runs real Git.
Seven steps. The teacher presses Next. The projector is the slide deck.

JOIN

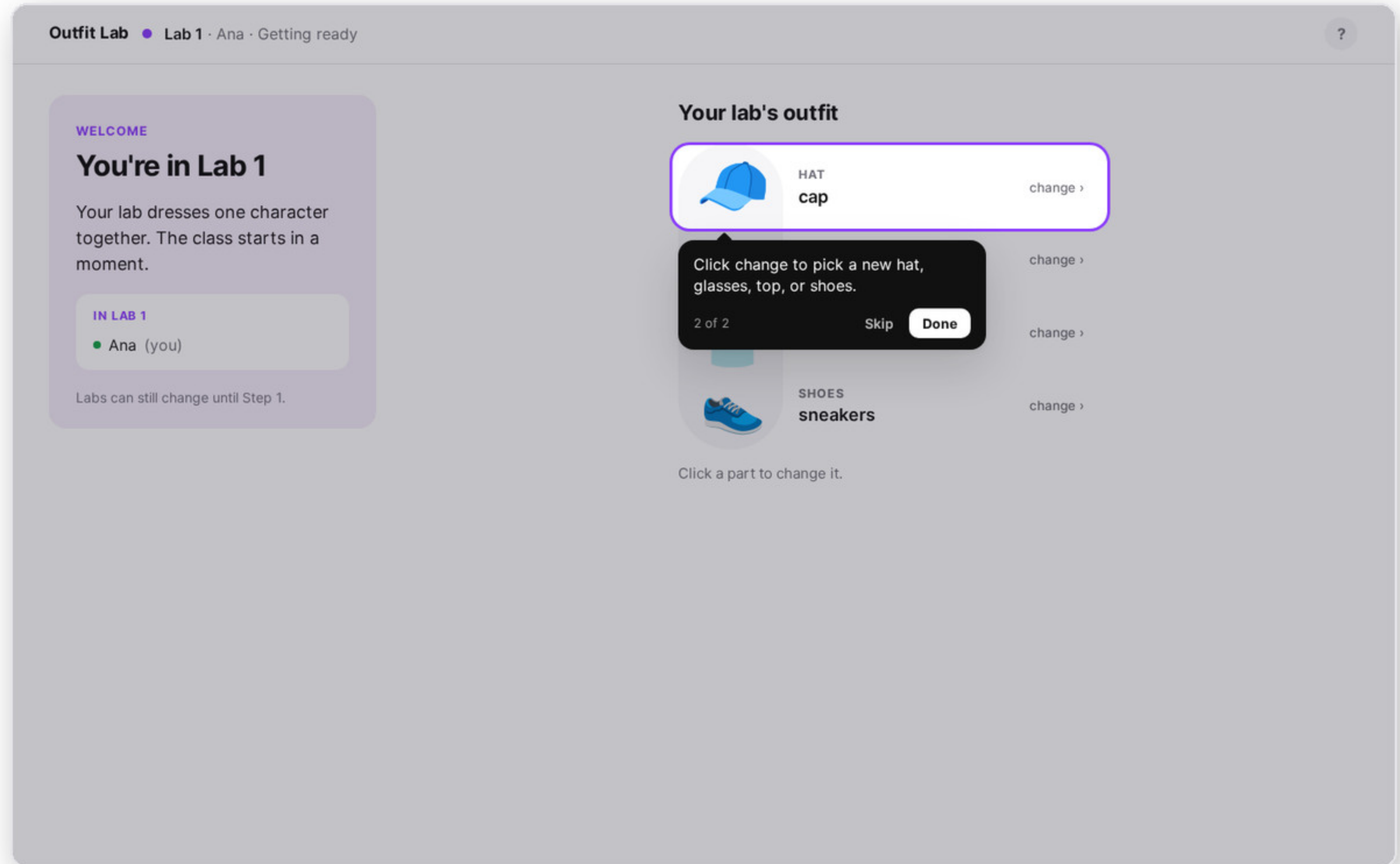
**Type a name.
That's it.**

LABS

The app picks each lab: about 4 people, two pairs. With 1 to 3 people, a practice lab plays along.

TOUR

A short tour points at the outfit and the change button. The ? replays it.



STEP 0 · CHAOS

Everyone, one outfit

PROBLEM

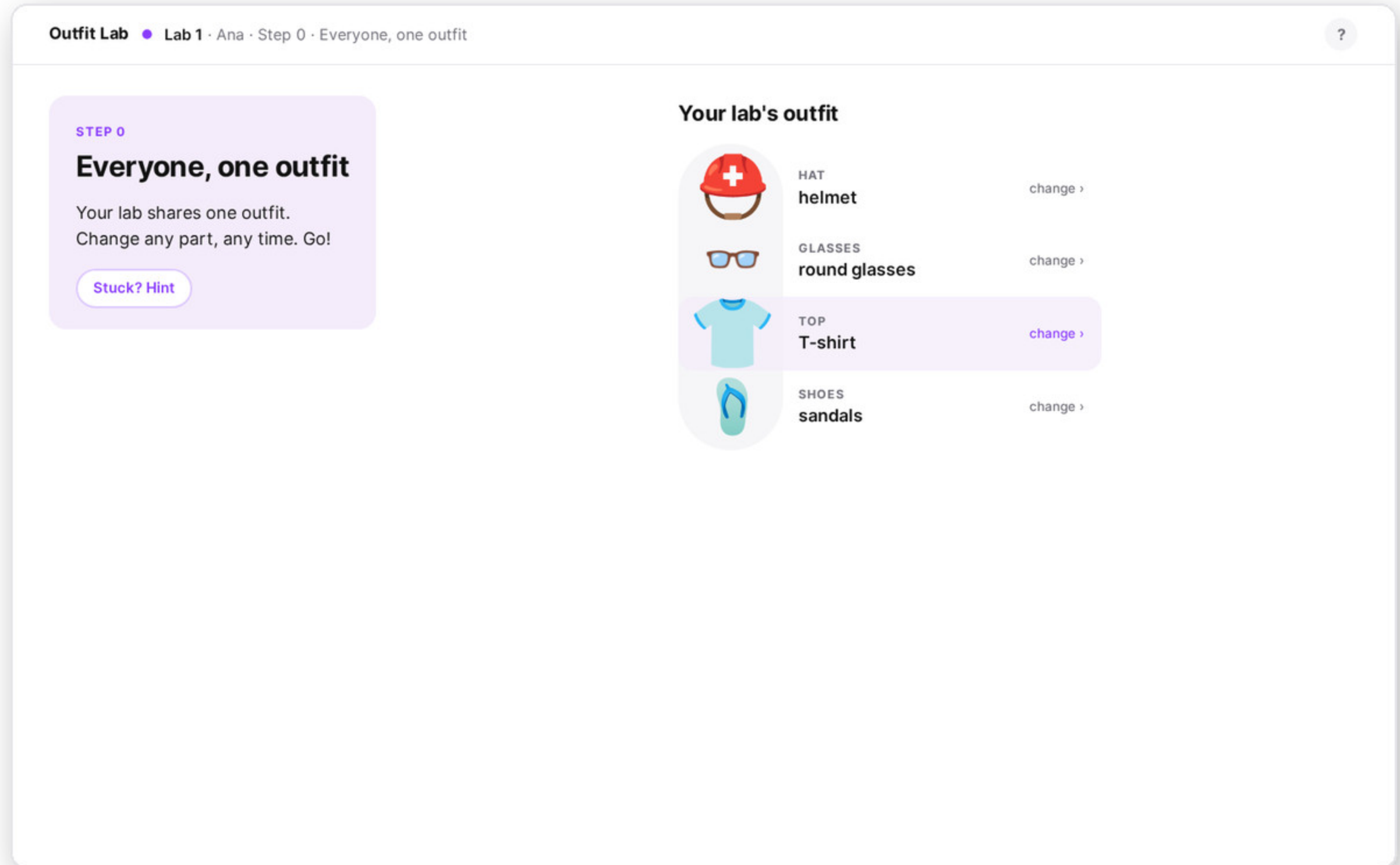
Everyone edits one outfit at once.
"Who changed the shoes?" Nobody knows.

IDEA

Save every version, with a name on it.

GIT

None yet. This is life without it.



STEP 1 · SAVE

Save every version

PROBLEM

Step 0's outfit is gone. Nobody can get it back.

IDEA

Every save is a new card: the whole outfit, the card before, your name. Cards never change.

GIT

`git commit`. The ID is a hash of the whole card, so any change gives a new ID. Following the arrows back is `git log`.

Outfit Lab • Lab 1 • Ana • Step 1 • Save every version

STEP 1

Save every version

Step 0's outfit is gone. From now on, every save makes a card. One at a time: change one part, then press **Save card**. Everyone saves once.

✓ Everyone saved a card (3/3)

Done ✓. Wait for the class.

Bonus • Click any card. Press **Show what Git stored**. Find the parent, the author and the message.

Discuss: What should each saved version hold?

Behind the door • what Git did

Shared draft

HAT
sun hat

change ›

GLASSES
goggles

change ›

TOP
coat

change ›

SHOES
sneakers

change ›

All saved

Save card

git commit

NEW

Your lab's cards

git log

2682fc3
Outfit Lab
Start

d4e2701
Ana
HAT: cap → sunhat

b001dd3
Lea
GLASSES: round → goggles

9540d95
Kim
TOP: tshirt → coat

← points to the card before

HOW GIT DOES IT

One technical card per tool

ON SCREEN

The command, then three rows: what it is, what it does, how Git does it. The question sits on top.

IN THE APP

Students see the same card. They answer the question and write a one-line takeaway.

STEP 1

One more line: Git stores the name and clock your laptop gives it. It checks neither.

STEP 1

Save every version

Why do arrows point back, never forward?

git commit

WHAT IT IS

a saved snapshot of the whole project.

WHAT IT DOES

records the exact state, who saved it, when, and the commit before it.

HOW GIT DOES IT

each file is stored as a blob, the folder as a tree. A commit object = tree ID + parent ID(s) + author + committer + message. Its ID is the SHA-1 of that object, so any change gives a new ID. Nothing is edited in place.

Git stores the name and clock your laptop gives it. It checks neither.

STEP 2 · TWO IDEAS

Try two ideas at once

PROBLEM

Two ideas, one draft: the pairs overwrite each other.

IDEA

Each pair gets its own sticky note, fancy or sporty. Saving moves only the note you are on.

GIT

`git switch -c fancy`. A branch is a tiny file holding one card's ID. HEAD, your pin, records which note you are on.

STEP 2

Try two ideas at once

Your lab already has one sticky note: main. Each pair makes its own note for its idea. Build on it and save.

YOUR MISSION

One of you: press **New sticky note**, keep the name **fancy**. The other: wait for it, then **Switch to fancy**. Then HAT → 🎩, TOP → 👔. **Save card**.

You're in Pair A with Kim ([change](#))

- ✓ fancy has 🎩 + 👔
- ✓ sporty has 👕 + 👟

Done ✓. Wait for the class.

Bonus · **Switch to main** and back. Watch the outfit change. Look, don't edit.

Behind the door

what Git did

Shared draft

You're on: **fancy**

	HAT top hat	change ›
	GLASSES goggles	change ›
	TOP shirt & tie	change ›
	SHOES sneakers	change ›

All saved

New sticky note

`git switch -c`

NEW

Switch to **main**

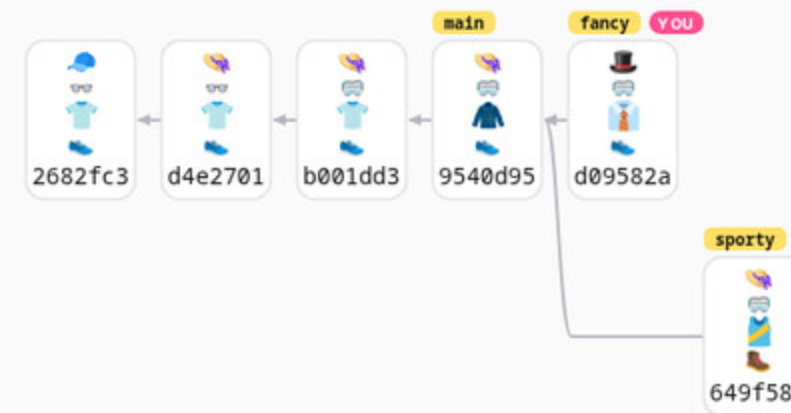
`git switch`

NEW

Save card

`git commit`

Your lab's cards `git log`



← points to the card before **main** sticky note **YOU** your pin (HEAD): the note you're on

STEP 3 · COMBINE

Make one outfit from both

PROBLEM

The client wants both ideas.

IDEA

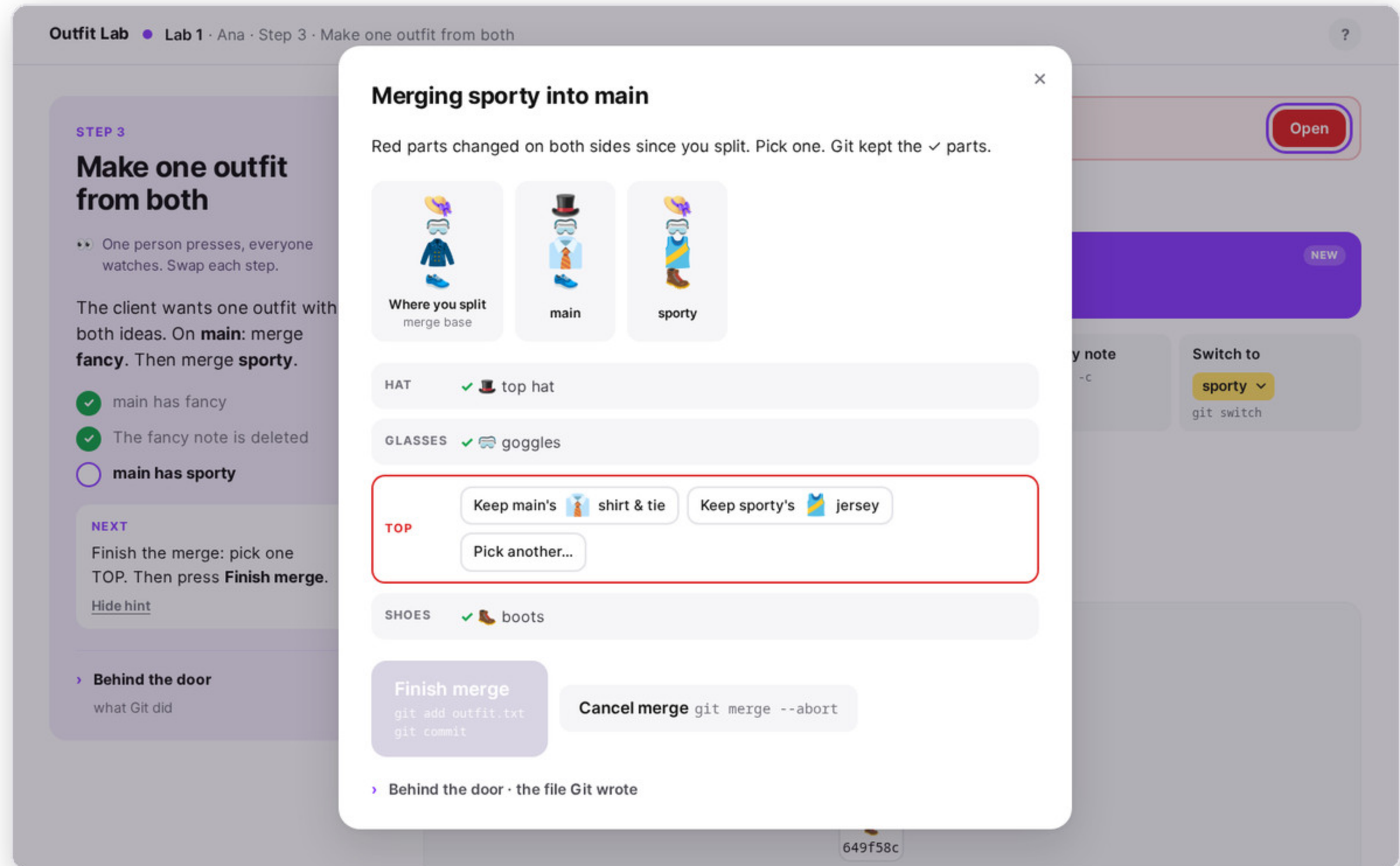
Compare each side with the card where they split. Changed on one side only: keep it.

GIT

`git merge`. fancy fast-forwards: only main's note moves. `git branch -d` deletes fancy. In sporty, TOP changed on both sides: a person picks.

ASK

Which cards were made on fancy? No way to tell. Git does not record the branch a commit was made on.



STEP 4 · SHARE

Put your outfit on the Wall

PROBLEM

The Wall: the class's shared copy, like GitHub. Another lab sent first. Yours was refused.

IDEA

Get the Wall's cards first: combine, or replay yours on top.

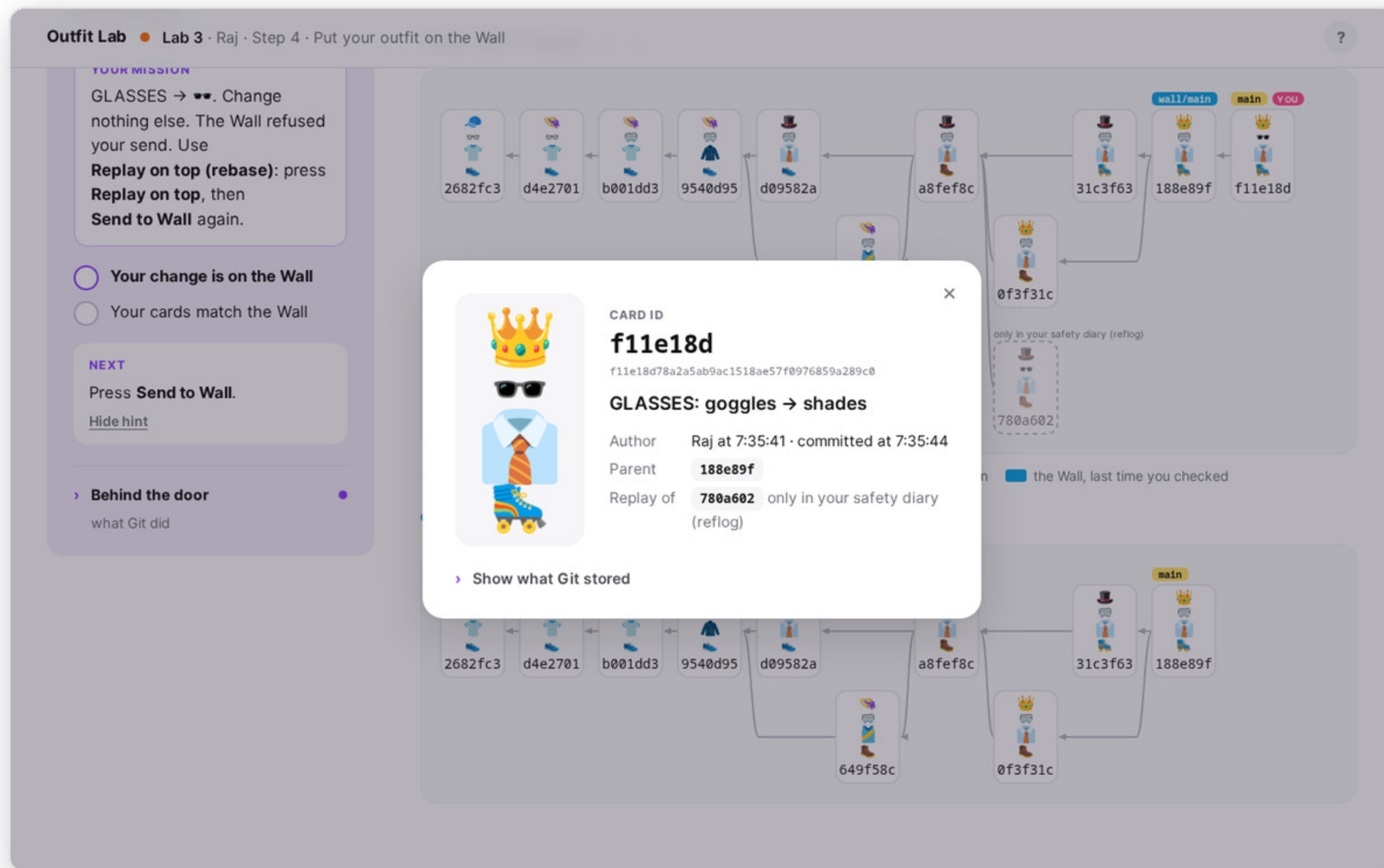
GIT

`git pull` = fetch + merge.

`git pull --rebase` replays each card on top. Kept: change, author, author time. New: parent, committer time, ID.

ON SCREEN

The dashed original is only in the safety diary (reflog).



STEP 4 · HOW GIT DOES IT

Merge and rebase, side by side

ON THE WALL

A merge card with two parents. Next to it, a straight line: a copy with a new ID.

PATH

One change's path to main, in bold, with this class's times: made, then on the Wall. The gap is the paper's code velocity.

ASK

Same change, author and author time: why a new ID? Its parent is new, so its snapshot is too. Its committer time is new. The ID is a hash of all of it.

STEP 4 Put your outfit on the Wall

The replayed card has the same change, author and author time. Why does it have a new ID?

git push / git pull

WHAT IT IS	sharing commits with a copy elsewhere (the Wall).
WHAT IT DOES	push sends your new commits and moves the remote branch. pull brings theirs in.

HOW GIT DOES IT copies exchange only the objects the other side lacks; IDs come from content, so copies agree without coordination. A push is accepted only if it is a fast-forward of the remote branch. Otherwise fetch first, then merge or rebase.

git rebase

WHAT IT IS	replaying your commits on top of another branch.
WHAT IT DOES	gives a straight history with no merge commit.

HOW GIT DOES IT for each of your commits, Git applies its change to the new base and writes a new commit. The new parent gives it a new ID. Author and author date are kept. The originals become unreachable and stay in your reflog for a while.

THE WALL · LAB 2'S PATH TO MAIN (BOLD)



- **Lab 1** 🧑‍🔬 · made 7:35 → on the Wall 7:35 · 0 s
- **Lab 2** 🧑‍🔬 · **made 7:35 → on the Wall 7:35 · 1 s**
The path ends at its own merge card, with two parents.
- **Lab 3** 🧑‍🔬 · made 7:35 → on the Wall 7:35 · 4 s
Its original card is not on the Wall. The path starts at a copy.

STEP 5 · UNDO

Oops: undo a shared mistake

PROBLEM

The Intern's "Tiny style fix" put a disguise on the Wall. Every lab has it too.

IDEA

Don't rip out a shared card. Add a card that undoes it.

GIT

`git revert` adds a fix card, so history only grows. `git reset` moves your note back: the Wall refuses the send, and the next pull brings the card back. `git reflog` lists where main has been.

Outfit Lab • Lab 2 • Priya • Step 5 • Oops: undo a shared mistake

STEP 5

Oops: undo a shared mistake

One person presses, everyone watches. Swap each step.

A 🤖 card reached the Wall. Remove it without breaking anyone's copy.

YOUR MISSION

Refused. Open the **Safety diary**, then press **Get & combine**. 🤖 back? Click it, press **Undo this card**, then send.

✔ You undid the 🤖 card yourselves

○ No 🤖 on the Wall or in your lab (Send to Wall)

Stuck? Hint

Behind the door

what Git did

Safety diary git reflog main

Every place main has been. Newest first. Moving a note deletes nothing. Click a line to open that card.

7:35 AM

🤖

Moved back • f11e18d
reset: moving to f11e18d

7:35 AM

🤖

Got the Wall's cards • ee36abe
merge wall/main: Fast-forward

7:35 AM

🤖

Got the Wall's cards • f11e18d
merge wall/main: Fast-forward

7:35 AM

🤖

Got & combined • 188e89f
merge wall/main: Merge made by the 'ort' strategy.

7:35 AM

🤖

Saved a card • 31c3f63
commit: SHOES: boots → skates

7:35 AM

🤖

Copied from the Wall • a8fef8c
clone: from the Wall

← points to the card before

sticky note

YOU your pin (HEAD): the note you're on

the Wall, last time you checked

The Wall

the class's shared copy

or move your note back.

New sticky note

git switch -c

Send to Wall

git push

Replay on top

git pull --rebase

31c3f63

188e89f

f11e18d

ee36abe

f31c

STEP 6 · CLEAN UP

The boss wants it clean

PROBLEM

"I want one clean card." Before the clean-up, the Wall knows who first added the boots.

IDEA

One new card after Start, with today's outfit, replaces the history.

GIT

Squash writes one new card.

`git push --force` moves the Wall onto it. `git gc --prune=now` deletes the cards no note leads to.

AFTER

Asked again, the Wall has no answer. Labs that kept the old cards still know.

STEP 6

The boss wants it clean

Who added the boots? Where does that answer still exist?

Squash + git push --force

WHAT IT IS

cleaning up history.

WHAT IT DOES

squash turns several commits into one. Force push makes the Wall accept it anyway.

HOW GIT DOES IT

squash writes one new commit whose tree is the final state. The originals, with their authors and times, become unreachable. `--force` skips the fast-forward check and moves the remote branch. `git gc` later deletes unreachable objects.

Who first added 🦶? Wall: not found · Lab 1: not found · Lab 2: Lea (Lab 1), 649f58c · Lab 3: Lea (Lab 1), 649f58c

The Wall's bin: 19 old cards, now 0.

THE PAPER

Flat history is data loss.

PAPER

Switching to Git: the Good, the Bad, and the Ugly. Just, Herzig, Czerwotka, Murphy. ISSRE 2016.

LIVED IT

After the fast-forward and the deleted note, no card says "fancy". The rebased original is not on the Wall. The squash dropped the cards and who made them.

ASK

One rule: the boss gets a clean history, and the auditor still knows who added the boots.

Tuesday's paper · Just, Herzig, Czerwotka, Murphy · ISSRE 2016

Switching to Git: the Good, the Bad, and the Ugly

GOOD

Cheap branches.
Local commits and reverts.
Developers prefer a flat history.

BAD

Fast-forward forgets which branch a change came from.
Rebase rewrites the commit; the patch can change too.
Squash drops the cards, even who made them.

UGLY

Microsoft traces each change's route to main: its integration path.
For Git, that tracing had to be redesigned from scratch.
Some loss cannot be recovered.

CODE VELOCITY IN THIS CLASS

- **Lab 1** 🧑‍🔬 · made 7:35 → on the Wall 7:35 · 0 s
- **Lab 2** 🧑‍🔬 · made 7:35 → on the Wall 7:35 · 1 s
- **Lab 3** 🧑‍🔬 · made 7:35 → on the Wall 7:35 · 4 s

After the squash, the Wall has none of these cards.

WHAT YOU LIVED

Step 3: after the fast-forward, you deleted fancy. No card says which cards were made on it.
Step 4: Rebase rewrites the change and gives it a new ID. The original card is not on the Wall.
Step 6: the squash dropped the cards, and who made them.
Step 6: the 🧑‍🔬 audit failed. The Wall no longer knows who added the boots.

Flat history is data loss.

WRAP

What you built

Cards never change.

Sticky notes move.

The Wall copies cards.

TAKE HOME

My Git in 7 lines: one takeaway per step, 0 to 6, editable, with Copy.

Outfit Lab • Lab 1 • Ana • What you built

WHAT YOU BUILT

Cards never change. Sticky notes move. The Wall copies cards.

My Git in 7 lines

Copy

- 0 Without saves, nobody knows who changed what.
- 1 A card is a full snapshot plus its parent.
- 2 A branch is a label on one card.
- 3 Merge = compare both sides with the card they share.
- 4 Push only moves the Wall forward.
- 5 Shared mistake: revert, don't reset.
- 6 Squash + force push: the Wall forgets who did what.

Your takeaways, in step order. Edit any line. An empty line copies the board's line.

What Lab 1 did

7

cards saved

2

merges

1

conflict solved

3

sends to the Wall

1

refused send

1

undo

Git ideas you used

- ✓ Save a card `git commit`
- ✓ Switch notes `git switch`
- ✓ Combine two ideas `git merge`
- ✓ Send to the Wall `git push`
- ✓ Get & combine `git pull --no-rebase`
- ✓ Undo with a fix card `git revert`
- ☐ Read the diary `git reflog`
- ✓ Make a sticky note `git switch -c`
- ✓ Slide a note forward `git merge`
- ✓ Solve a conflict `git merge + git commit`
- ✓ Get refused `git push`
- ☐ Replay on top `git pull --rebase`
- ☐ Move a note back `git reset`
- ✓ Replace the Wall `git push --force`

TEACHERS

Press Next.
That's the class.

NEXT

One button walks 19 scenes in 77 minutes. A clicker works. The projector follows: it is the slide deck.

SCRIPT

Each scene: what to say and do, the one question with its answer folded, and live answers.

LABS

One tile per lab: goals, outfit and status. In Step 4, its way: merge or rebase.

Outfit Lab · Teacher9 people · 9 online

Join QRStart presenting

SCENE 12 OF 19 · PLAN 0:45

Step 4 · How Git does it0:01 / 5:00 · 44 min ahead

SAY

The Wall only moves forward, so a refused lab gets the Wall's cards first. Combine makes a merge card with two parents (Step 3 made one too). Replay on top makes a straight line: a copy of the card with the same change, author and author time, but a new parent, snapshot, committer time and ID.

DO

Point at the marked path on the Wall: when the card was made, and when it reached the Wall. These are this class's times.

ASK

The replayed card has the same change, author and author time. Why does it have a new ID?

Show on projectorAnswer

4/9 answeredShow answers on projectorwith names

Lea Lab 1 · Its parent is new, so the hash is new.

Sam Lab 2 · The ID hashes the parent ID too.

Kim Lab 1 · A new parent means a new card.

Ola Lab 3 · Same change, different card: the parent changed.

Takeaways written: 9/9

BOARD

4. The Wall only moves forward. Behind? Combine (merge), or replay on top (rebase: new IDs).

PATHS

Lab 1👑made 7:35 → on the Wall 7:35 · 0 s

Lab 2👑made 7:35 → on the Wall 7:35 · 1 s

Lab 3👑made 7:35 → on the Wall 7:35 · 4 s · starts at a copy

Sabotage: the Intern's👑 card

Next: Step 5 · Oops: undo a shared mistakeBack

Lab 13 people

👑

✓ Your change is on the Wall

✓ Your cards match the Wall

Done

Lab 23 people

👑

✓ Your change is on the Wall

✓ Your cards match the Wall

Done

Way: Combine (merge)

Lab 33 people

👑

✓ Your change is on the Wall

✓ Your cards match the Wall

Done

Way: Replay on top (rebase)

Details

ON THE PROJECTOR NOW

STEP 4Put your outfit on the Wall

The replayed card has the same change, author and author time. Why does it have a new ID?

git push / git pull

what it is: sharing contents with a copy elsewhere (the Wall).

what it does: push sends your new contents and makes the remote branch; pull brings them in.

what it selects: copies exchange only the objects the other side lacks. (On same team content, all copies agree without coordination. A push is accepted only if it is a fast-forward of the remote branch. Otherwise fetch first, then merge or rebase.)

git rebase

what it is: replaying your contents on top of another branch.

what it does: gains a straight history with no merge content.

what it selects: for each of your parents, GR applies its change to the new base and writes a new commit. The new parent gives it a new ID. Author and author date are kept. The originals become unresolvable and stay in your reflog for a while.

THE WALL · LAB 2'S PATH TO BAHN (8:02)

Lab 1👑made 7:35 → on the Wall 7:35 · 0 s

Lab 2👑made 7:35 → on the Wall 7:35 · 1 s

Lab 3👑made 7:35 → on the Wall 7:35 · 4 s

The push ends at its own merge card, with two parents. The original card is not on the Wall. The push starts at a copy.

NEXT

Step 5 · Oops: undo a shared mistake

A👑 card reached the Wall. Remove it without breaking anyone's copy.

Puts the Intern's👑 card on the Wall, if you haven't yet. Each lab with nothing unsent gets it (a fast-forward).

REHEARSE ALONE

A whole class, with bots

BOTS

Details → Rehearse with bots. 2 to 12 bots join like students and follow their own hints.

SPEED

Real time, 5× or 20× faster. You press only Next.

YOU TOO

Open the student link in another window and join. Your lab then waits for you.

Outfit Lab · Teacher

9 people · 9 online

Rehearsal · 9 bots · 20× faster

Join QR

Start presenting

SCENE 11 OF 19 · PLAN 0:37

Step 4 · Put your outfit on the Wall

0:01 / 8:00 · 37 min ahead

SAY

The Wall is the class's shared copy, like GitHub. Your lab already has a full copy of it (that is git clone). It starts as Lab 1's outfit. Put your outfit on the Wall. The first lab there wins.

DO

Don't ask first: students hit the refusal themselves. Each refused lab gets its way in the app: Combine (merge) or Replay on top (rebase).

Labs done: 3/3

Next: Step 4 · How Git does it

Back

Lab 1

3 people



✓ Your change is on the Wall

✓ Your cards match the Wall

Done

Lab 2

3 people



✓ Your change is on the Wall

✓ Your cards match the Wall

Done

Way: Combine (merge)

Lab 3

3 people



✓ Your change is on the Wall

✓ Your cards match the Wall

Done

Way: Replay on top (rebase)

Details

ON THE PROJECTOR NOW

STEP 4 Put your outfit on the Wall

7:59

Make your change, Save card, Send to Wall. The first lab there wins.



Lab 1

✓ Your change is on the Wall

✓ Your cards match the Wall

Lab 2

✓ Your change is on the Wall

✓ Your cards match the Wall

Lab 3

✓ Your change is on the Wall

✓ Your cards match the Wall

NEXT

Step 4 · How Git does it